Cryptology

LAB Assignment 4

Name: Vivek B S

Roll No: 261100690034

Subject: Cryptology

Level 1 — File Handling & String Operations

Question 1: Create and Populate cryptology.txt with Five Statements

Python Code:

```
file = open("cryptology.txt", "w")
for idx in range(1, 6):
    stmt = input(f"Enter statement {idx}: ")
    file.write(stmt + "\n")
file.close()
print("Successfully written statements to cryptology.txt")
```

Question 2: Read and Display Complete Contents of cryptology.txt

Python Code:

```
file = open("cryptology.txt", "r")
content = file.read()
print(content)
file.close()
```

Question 3: Read cryptology.txt Line-by-Line with Line Numbers

Python Code:

```
file = open("cryptology.txt", "r")
for line_no, line in enumerate(file, start=1):
    print(f"{line_no} {line.strip()}")
file.close()
```

Question 4: Store Line-Numbered Output in cryptology_no.txt

Python Code:

```
input_f = open("cryptology.txt", "r")
output_f = open("cryptology_no.txt", "w")

for line_no, line in enumerate(input_f, start=1):
    formatted_line = f"{line_no} {line.strip()}"
    output_f.write(formatted_line + "\n")

input_f.close()
output_f.close()
print("Numbered records stored in cryptology_no.txt")
```

Question 5: Determine Lines, Words, and Characters in cryptology.txt

Python Code:

```
file = open("cryptology.txt", "r")
content = file.read()
file.close()

lines_cnt = len(content.splitlines())
words_cnt = len(content.split())
chars_cnt = len(content)

print(f"Number of lines      : {lines_cnt}")
print(f"Number of words      : {words_cnt}")
print(f"Number of characters : {chars_cnt}")
```

Question 6: Count Character Types in cryptology_no.txt

Python Code:

```
file = open("cryptology_no.txt", "r")
content = file.read()
file.close()

upper = lower = digits = spaces = special = 0

for ch in content:
    if ch.isupper():
        upper += 1
    elif ch.islower():
        lower += 1
    elif ch.isdigit():
        digits += 1
    elif ch == ' ':
        spaces += 1
    elif ch != '\n':
        special += 1

print(f"Uppercase letters: {upper}")
print(f"Lowercase letters: {lower}")
print(f"Digits          : {digits}")
print(f"Spaces          : {spaces}")
print(f"Special chars   : {special}")
```

Question 7: Search for User-Accepted Word in cryptology.txt

Python Code:

```
file = open("cryptology.txt", "r")
text_data = file.read().lower()
file.close()

target = input("Enter word to search: ").lower()
if target in text_data:
    print(f"The word '{target}' was found in cryptology.txt.")
else:
    print(f"The word '{target}' was not found.")
```

Question 8: Display Lines Containing the Word 'message'

Python Code:

```
file = open("cryptology.txt", "r")
for line in file:
    if "message" in line.lower():
        print(line.strip())
file.close()
```

Question 9: Normalize Text to Uppercase and Store in normalized.txt

Python Code:

```
input_f = open("cryptology.txt", "r")
output_f = open("normalized.txt", "w")

for line in input_f:
    output_f.write(line.upper())

input_f.close()
output_f.close()
print("Normalized text saved to normalized.txt")
```

Question 10: Print Lines Containing Only Alphabetic Characters

Python Code:

```
file = open("cryptology_no.txt", "r")
for line in file:
    clean_line = line.strip()
    parts = clean_line.split(maxsplit=1)
    if len(parts) > 1 and parts[1].isalpha():
        print(clean_line)
file.close()
```

Question 11: Get String from User and Reverse It

Python Code:

```
user_text = input("Enter a string: ")
print("Reversed string:", user_text[::-1])
```

Question 12: Reverse String Content Line-by-Line from File

Python Code:

```
file = open("cryptology.txt", "r")
for line in file:
    clean_line = line.strip()
    print(f"Original: {clean_line}")
    print(f"Reversed: {clean_line[::-1]}\n")
file.close()
```

Question 13: Simulate the UNIX 'tac' Command in Python

Python Code:

```
file = open("cryptology.txt", "r")
lines = file.readlines()
file.close()

for line in reversed(lines):
    print(line.strip())
```

Question 14: Alternative Pipeline: input.txt to processed.txt

Python Code:

```
file = open("input.txt", "w")
for i in range(1, 6):
    file.write(input(f"Enter line {i}: ") + "\n")
file.close()

file = open("input.txt", "r")
raw_data = file.read()
file.close()

print("Lines             :", len(raw_data.splitlines()))
print("Words             :", len(raw_data.split()))
print("Alphabetic        :", sum(ch.isalpha() for ch in raw_data))
print("Digits            :", sum(ch.isdigit() for ch in raw_data))
print("Spaces            :", sum(ch == ' ' for ch in raw_data))
print("Special chars     :", sum(not ch.isalnum() and ch not in " \n" for ch in raw_data))

processed_text = raw_data.upper()
out_file = open("processed.txt", "w")
out_file.write(processed_text)
out_file.close()
```

```
print("\nContents of processed.txt:")
print(processed_text)
```

Level 2 — Pattern Matching with Regular Expressions

Question 15: Identify and Display All IPv4-Address-Like Patterns

Python Code:

```
import re

file = open("security_data.txt", "r")
logs = file.read()
file.close()

ip_regex = r"\b(?:\d{1,3}\.){3}\d{1,3}\b"
matched_ips = re.findall(ip_regex, logs)

print("Identified IPv4 Addresses:")
for ip in matched_ips:
    print(ip)
```

Question 16: Identify and Display All Email Address Patterns

Python Code:

```
import re

file = open("security_data.txt", "r")
logs = file.read()
file.close()

email_regex = r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b"
matched_emails = re.findall(email_regex, logs)

print("Identified Email Addresses:")
for email in matched_emails:
    print(email)
```

Question 17: Extract Session IDs Following SEC-YYYY-NNNN Format

Python Code:

```
import re

file = open("security_data.txt", "r")
logs = file.read()
file.close()

session_regex = r"\bSEC-\d{4}-\d{4}\b"
matched_sessions = re.findall(session_regex, logs)

print("Extracted Session IDs:")
for sid in matched_sessions:
    print(sid)
```

Question 18: Identify Hexadecimal Sequences (0-9, a-f, A-F)

Python Code:

```
import re

file = open("security_data.txt", "r")
logs = file.read()
```

```
file.close()

hex_regex = r"\b[0-9A-Fa-f]+\b"
hex_matches = re.findall(hex_regex, logs)

print("Extracted Hexadecimal Sequences:")
for hex_val in hex_matches:
    print(hex_val)
```

Question 19: Display Lines Containing 'failed' (Case-Insensitive)

Python Code:

```
import re

file = open("security_data.txt", "r")
for line in file:
    if re.search(r"\bfailed\b", line, re.IGNORECASE):
        print(line.strip())
file.close()
```

Question 20: Extract Port Number from 'Port 443 connection established'

Python Code:

```
import re

file = open("security_data.txt", "r")
logs = file.read()
file.close()

port_regex = r"\bPort\s+(\d+)\s+connection\s+established\b"
extracted_ports = re.findall(port_regex, logs, re.IGNORECASE)

print("Captured Port Numbers:")
for port in extracted_ports:
    print(port)
```

Question 21: Read security_data.txt and Extract Patterns to pattern_results.txt

Python Code:

```
import re

file = open("security_data.txt", "r")
log_data = file.read()
file.close()

patterns = {
    "IP-address-like patterns": r"\b(?:\d{1,3}\.){3}\d{1,3}\b",
    "Email addresses": r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
    "Session IDs": r"\bSEC-\d{4}-\d{4}\b",
    "Hexadecimal sequences": r"\b[0-9A-Fa-f]+\b"
}

output_f = open("pattern_results.txt", "w")

for title, pattern in patterns.items():
    matches = re.findall(pattern, log_data)
    output_f.write(f"{title}:\n")
    for match in matches:
        output_f.write(f"{match}\n")
    output_f.write("\n")

output_f.close()
print("Patterns successfully extracted and stored in pattern_results.txt")
```